GPU Histogram & Example-Management Variants

Oblique decision-tree training — kernel inventory & measured progression. Dataset: HIGGS1 (11,000,000 rows, 28 features), 2 trees.

1 · Background layout lineages (pre-existing baselines)

Approach	Build config	Example management	Projection kernel	Histogram kernel
SRDW (sort per depth)	*default*	sort examples per node → `d_selected_examples`	`ColumnAddProjectionKernel_SRDW_1_2D`	`BuildHistogramRandomKernel_2D`
DRSW	`--config=drsw`	`ComputeDRSWArrays` (UpperBound → node_ids/offsets)	`ColumnAddProjectionKernel_DRSW_1`	`BuildHistogramRandomKernel_2D`

2 · NODE_ID_DRSW lineage — `--config=node_id_drsw`

Persistent `d_node_ids[num_rows]`; per-depth derives offsets instead of re-sorting. Projection is always `NodeIdDRSW_ProjectionOnly`; split is always `NodeIdDRSW_SplitUpdate`. Variation lives in two stages:

Stage	Kernel	Role	Selected by
Compute offsets	`..._ComputeOffsetsKernel_Smem_OldAtomicAdd`	per-thread shared-mem atomicAdd ranking	`--config=no_two_pass` (+ A/B)
	`..._ComputeOffsetsKernel_Smem`	warp-ranked single pass	A/B only (`COMPUTE_OFFSETS_AB=1`)
	`..._ComputeOffsets_CountKernel` + `_ScatterKernel`	two-pass: count → prefix-sum → scatter	`TWO_PASS` (**default on**)
	`..._ComputeOffsetsKernel_Global`	global-atomic fallback	`num_nodes > 256`
	`..._ComputeOffsetsOnlyKernel`	offsets only, no `d_selected_examples`	`--config=offsets_only`
Split update	`..._SplitUpdateKernel_GlobalAtomic`	per-row global atomicAdd	`--config=split_global_atomic`
	`..._SplitUpdateKernel_SmemReduce`	shared-mem block reduction, flush once/block	`SPLIT_SMEM` (**default on**)

3 · Partition / SplitExamples kernels (example → child reassignment)

The `partition` timing bucket. Two separate code paths depending on lineage.

Lineage	Host wrap	Device kernel(s)	Selected by
SRDW / DRSW	`SplitExamplesInPlaceKernel_all_depth_No_D2H_fused_wrap`	`DirectScatterFusedAllDepthKernel_1D`	`--define=d2d=dir`
SRDW / DRSW		`StablePartitionCountKernel` + `StablePartitionScatterKernel`	default (stable tw pass)
NODE_ID_DRSW	`NodeIdDRSW_SplitUpdate` → two kernels, see sub-table below		`node_id_drsw`

NODE_ID_DRSW split — `NodeIdDRSW_SplitUpdate` (reads node_id + offset)

Version	Kernel	Accumulates child + per-class counts	Selected by
shared-memory	`NodeIdDRSW_SplitUpdateKernel_SmemReduce`	block-local smem; one `atomicAdd` per key per block (flush once)	`SPLIT_SMEM` — **default on**
global atomic	`NodeIdDRSW_SplitUpdateKernel_GlobalAtomic`	per-row global `atomicAdd`	`--config=split_global_atomic`

Auto-fallback: even with `SPLIT_SMEM` on, the global-atomic kernel is used when the required shared memory (`num_children × (1 + num_classes) × 4 B`) exceeds the device per-block cap (wide trees / many classes). A/B harness: `SPLITUPDATE_AB=1` times GLOBAL vs SMEM head-to-head.

All in `oblique_apply_projection.cu` except the NODE_ID_DRSW split (`node_id_drsw.cuh`). Older SplitExamples kernels (`ComputeSplitFlagsKernel`, `ScatterSplitKernel`, `…AllDepthKernel`, `CompactSplitResultsKernel`) live only in `*_bak1/_bak2.cu` and are **not compiled**.

4 · Histogram-kernel inventory

Kernel	Value / label access	Selected by
`BuildHistogramRandomKernel`	1D original	*superseded*
`BuildHistogramRandomKernel_2D`	projected coalesced; **label gathered** `d_labels[d_selected_examples[...]]`	SRDW/DRSW base, or `--config=nid_srdw_hist`
`BuildHistogramRandomKernel_2D_WarpAgg`	same, warp-aggregated atomics	runtime `HIST_WARPAGG=1`
`BuildHistogramDRSWKernel`	iterate all rows; **node_id + offset**; global atomics for non-leader nodes	`node_id_drsw` default / `offsets_only`
`BuildHistogramCompactKernel`	like DRSW but over compacted active rows	implemented, **not wired** (experimental)
`BuildHistogramSRDWLabelsKernel` ★ **new**	projected coalesced; **label coalesced** `d_labels_srdw[row_start+r]`	`--config=srdw_label_hist`

5 · Measured progression — `build_hist` on HIGGS1 (worst → best)

Sum over depths 1–9, averaged across 2 trees. All three are NODE_ID_DRSW builds.

Rank	Build config combo	Histogram kernel	Label source	build_hist sum
①	`node_id_drsw` + `nid_srdw_hist`	`BuildHistogramRandomKernel_2D`	scattered gather, per-projection, grows with depth	**84.8 ms**
②	`node_id_drsw` + `offsets_only`	`BuildHistogramDRSWKernel`	`d_labels[r]` coalesced, but global atomics	**48.2 ms**
③ ★	`node_id_drsw` + `offsets_only` + `srdw_label_hist`	`BuildHistogramSRDWLabelsKernel`	`d_labels_srdw[...]` coalesced, block-per-node	**17.4 ms**

Speedups: ①→② = 1.76× · ②→③ = 2.77× · ①→③ = 4.9× overall. Variant ③ is also flat (~1.9 ms/depth) while ① climbs 3 → 15 ms with depth.
Correctness: tree-0 per-depth node counts of ③ match the baseline exactly.

6 · Not yet measured on HIGGS1 (built-in A/B harnesses)

- **SRDW base** and **DRSW base** histograms — separate `--config` builds.
- **ComputeOffsets** OLD vs WARP vs 2PASS — run any node_id build with `COMPUTE_OFFSETS_AB=1`.
- **SplitUpdate** GLOBAL vs SMEM — `SPLITUPDATE_AB=1`.
- **Histogram** ORIG vs WARPAGG — `HIST_AB=1`.

Per-depth timings print to stderr under `--config=verbose_depth` (field `hist=`) and to `per_depth_timing.csv`. Build command for the new variant:
`bazel build //examples:oblique_apply_projection_main --config=bfs_only --config=use_gpu --config=verbose_depth --config=node_id_drsw`